UNIVERSIDAD - FACULTAD DE INGENIERIA Y CIENCIAS BASICAS
PROGRAMA DE INGENIERIA DE SOFTWARE

CURSO: SISTEMAS OPERATIVOS

INFORME TECNICO INTEGRAL DE PROYECTO
Planificacion de CPU y Gestion de Memoria con Paginacion

Simulacion Ciclo a Ciclo de Procesos (FCFS, SJF, Round Robin),
Overhead de Cambio de Contexto, Evaluacion de Equidad e Inanicion,
y Paginacion Pura con Traduccion MMU de 4 Pasos (Reemplazo FIFO vs. LRU)

INTEGRANTES DEL EQUIPO (GRUPO DE PROYECTO):

- [Estudiante 1 - Codigo: _____]
- [Estudiante 2 - Codigo: _____]
- [Estudiante 3 - Codigo: _____]
- [Estudiante 4 - Codigo: _____]

DOCENTE EVALUADOR: [Nombre del Docente del Curso]
FECHA DE ENTREGA: Noviembre de 2026
STACK TECNICO: Python 3.11, Matplotlib, Pandas (Simulacion Pura sin Frameworks)

1. Descripcion del Problema

En los sistemas computacionales modernos, la Unidad Central de Procesamiento (CPU) y la Memoria Principal (RAM) constituyen los dos recursos fisicos mas criticos y limitados. Dado que el numero de tareas concurrentes excede ampliamente la cantidad de nucleos fisicos y la capacidad contigua de la memoria, el sistema operativo (SO) enfrenta el desafio de **multiplexar tanto el tiempo de CPU como el espacio de memoria** de forma transparente, segura y determinista.

A nivel de procesador, la ausencia de una politica de planificacion adecuada provoca patologias severas como el **Efecto Convoy**, degradacion de interactividad y sobrecarga administrativa (*overhead*) por conmutaciones de contexto. En paralelo, a nivel de memoria, la asignacion contigua tradicional genera **fragmentacion externa** (memoria libre dispersa en huecos inservibles). Para superarlo, el SO implementa **paginacion**, dividiendo el espacio virtual en paginas y la RAM en marcos fijos, lo cual elimina la fragmentacion externa pero introduce **fragmentacion interna** y la necesidad de politicas optimas de reemplazo de paginas (FIFO vs. LRU) ante fallos de pagina (*page faults*).

2. Objetivo del Proyecto

Objetivo General: Disenar, implementar y evaluar un simulador ciclo a ciclo (reloj discreto *tick-by-tick*) en Python 3 que integre en un unico motor de ejecucion la **planificacion de CPU** (FCFS, SJF no expropiativo, Round Robin) y la **gestion de memoria con paginacion pura** (MMU con traduccion en 4 pasos, calculo de fragmentacion y reemplazo FIFO vs. LRU), evaluando cuantitativamente rendimiento, equidad, sobrecarga y aciertos de memoria sobre tres escenarios reproducibles.

Objetivos Especificos:

- Modelar el Bloque de Control de Proceso (PCB) bajo cinco estados formales: NUEVO, LISTO, EJECUCION, BLOQUEADO y FINALIZADO, con log de transiciones auditables.
- Incorporar el costo temporal de cambio de contexto (*costo_cs*) asegurando consistencia matematica estricta (WT = RT en algoritmos no expropiativos sin E/S).
- Implementar la Unidad de Manejo de Memoria (MMU) simulada con el procedimiento canónico de 4 pasos para traducir direcciones virtuales a fisicas.
- Contrastar experimentalmente las estrategias de reemplazo de paginas FIFO y LRU, demostrando la ventaja de LRU ante patrones de acceso con localidad temporal.
- Cuantificar la fragmentacion interna (memoria asignada - memoria necesitada) y constatar la ausencia de fragmentacion externa en paginacion.

3. Fundamentacion del Mecanismo Seleccionado

La coexistencia coordinada de la planificacion de CPU a corto plazo y la paginacion en memoria responde al principio clasico del **Ciclo de Rafagas CPU-E/S** (Silberschatz et al., 2018). Los procesos alternan entre computo en procesador y transferencia de datos de E/S. Un planificador eficaz solapa estas fases: mientras un proceso espera E/S en estado BLOQUEADO, la CPU se cede a otro proceso listo, elevando el aprovechamiento del hardware.

Asimismo, la paginacion desacopla el espacio de direcciones logico del espacio fisico: el proceso percibe una memoria virtual contigua, mientras la MMU mapea dinamicamente sus paginas a marcos fisicos no contiguos en la RAM. Esto erradica la compactacion pesada de memoria y, mediante la carga bajo demanda, permite ejecutar procesos cuya demanda virtual supera la memoria fisica disponible.

4. Diseno de la Solucion

La arquitectura del simulador es de **tiempo discreto determinista (tick-by-tick)**. Cada avance del reloj entero (*t = 0, 1, 2, ...*) orquesta las admisiones, la ejecucion de instrucciones, la traduccion de direcciones de memoria y el avance de los dispositivos de E/S.

4.1. PCB, Maquina de Estados y Modulos

El ciclo de vida del proceso implementa una maquina de estados finitos:
• **NUEVO -> LISTO:** Admision del proceso. Se crea su Tabla de Paginas calculando *num_paginas = ceil(memoria_necesitada / tamano_pagina)*.
• **LISTO -> EJECUCION:** Despacho a CPU tras el intervalo de cambio de contexto (*costo_cs*).
• **EJECUCION (Acceso a Memoria):** En cada tick de CPU, la MMU traduce la direccion virtual. Si la pagina no esta en RAM, ocurre un **fallo de pagina**, cargando la pagina en un marco libre o aplicando reemplazo (FIFO / LRU).
• **EJECUCION -> BLOQUEADO:** Transicion voluntaria por solicitud de E/S al completar una rafaga de CPU intermedia.
• **BLOQUEADO -> LISTO:** Desbloqueo al finalizar el conteo de ciclos de E/S.
• **EJECUCION -> LISTO:** Desalojo preventivo por expiracion de quantum en Round Robin.
• **EJECUCION -> FINALIZADO:** Al concluir sus rafagas, se liberan todos los marcos fisicos que el proceso ocupaba en RAM.

El sistema se organiza en 6 archivos modulares: pcb.py (PCB y estados), planificadores.py (algoritmos CPU), memoria.py (marcos, tablas, MMU y reemplazo), simulador.py (reloj y colas), metricas.py (cálculos matemáticos) y visualizador.py (graficación automatizada).

5. Algoritmos, Estrategias o Configuraciones Comparadas

A. Planificacion de CPU:
• **FCFS (First-Come, First-Served):** Cola FIFO pura no expropiativa. Atiende por orden de llegada; desempate por menor PID.
• **SJF (Shortest Job First No Expropiativo):** Selecciona el proceso con menor duracion en su rafaga actual de CPU. Desempate: 1) menor tiempo de llegada (FCFS local); 2) menor PID.
• **Round Robin (RR con quantum q):** Expropiativo por tiempo. Rota en rodajas de *q* ticks. En caso de colision simultanea entre fin de quantum y nueva llegada, se encola primero el proceso entrante para favorecer su tiempo de respuesta (RT).

B. Gestion de Memoria (Paginacion Pura y Reemplazo):
• **Procedimiento de Traduccion MMU (4 Pasos de las Diapositivas):**
1. *numero_pagina = direccion_virtual // tamano_pagina*
2. *desplazamiento = direccion_virtual % tamano_pagina*
3. Consultar tabla_paginas: si no esta en RAM -> fallo de pagina y seleccion de victima (FIFO / LRU).
4. *direccion_fisica = marco * tamano_pagina + desplazamiento*.
• **FIFO (First-In, First-Out):** Reemplaza la pagina que lleva mas tiempo cargada en la RAM (menor *tiempo_carga*).
• **LRU (Least Recently Used):** Reemplaza la pagina menos recientemente accedida (menor *ultimo_acceso*).

6. Escenarios y Metricas de Prueba

Se estructuraron tres escenarios reproducibles con parametros exactos (almacenados en resultados/logs/ y resultados/metricas/):

Escenario	Objetivo Experimental	Procesos y Parametros de Rafaga	Memoria Asignada (4 KB/pag)
1. Carga Ligera (con E/S)	Linea base, transicion a BLOQUEADO, sobrecarga	P1 (Llegada 0, CPU 3, IO 2, CPU 2) P2 (Llegada 1, CPU 4) P3 (Llegada 2, CPU 2, IO 1, CPU 2) P4 (Llegada 4, CPU 3)	P1: 6000 B (2 pags) P2: 4096 B (1 pag) P3: 7500 B (2 pags) P4: 3000 B (1 pag) Total: 6 pags en RAM 32 KB
2. Carga Mixta (Efecto Convoy)	Demostrar efecto convoy en CPU y comparar FIFO vs LRU	P1 (Llegada 0, CPU 20, IO 5, CPU 3) P2 (Llegada 1, CPU 2); P3 (Llegada 2, CPU 2) P4 (Llegada 3, CPU 1); P5 (Llegada 4, CPU 2)	P1: 35000 B (9 pags, bucle Silberschatz P2, P3, P5: 5000 B (2 pags c/u) P4: 3000 B RAM limitada: 4 marcos (16 KB)
3. Alta Concurrencia y Sensibilidad q	Trade-off de quantum en CPU y comportamiento de memoria	P1 (Llegada 0, CPU 8); P2 (Llegada 1, CPU 4); P3 (Llegada 2, CPU 12); P4 (Llegada 3, CPU 3); P5 (Llegada 5, CPU 6); P6 (Llegada 6, CPU 2); P7 (Llegada 8, CPU 10); P8 (Llegada 9, CPU 5)	P1: 14 KB; P2: 10 KB; P3: 15 KB; P4: 9 KB; P5: 12 KB; P6: 8.5 KB; P7: 11.5 KB; P8: 11 KB Total: 37 pags en RAM 32 KB (8 marcos

6.1. Formulas Matematicas de las Metricas

• **Turnaround Time (TAT):** *TAT_i = T_fin,i - T_llegada,i*; Promedio: *TAT_prom = (1/N) * sum(TAT_i)*.
• **Waiting Time (WT):** Ciclos acumulados en LISTO: *WT_i = TAT_i - T_CPU,i - T_IO,i*.
• **Response Time (RT):** Latencia hasta primer despacho: *RT_i = T_primer_despacho,i - T_llegada,i*.
• **Equidad (Desv. Estandar de WT):** *sigma_WT = raiz[(1/N) * sum((WT_i - WT_prom)^2)]*.
• **Inanicion Estadistica:** Alerta si ratio *WT_i / T_CPU,i >= 4.0* y/o *WT_i > WT_prom + 1.5 * sigma_WT*.
• **Tasa de Aciertos y Fallos de Memoria:** *Hit Rate (HR) = (Aciertos / Total Accesos) * 100%*; *PFR = 100% - HR*.
• **Fragmentacion Interna:** *IF_i = (Paginas_Asignadas_i * Tamano_Pagina) - Memoria_Necesitada_i*. Total = sum(IF_i).

7. Resultados Obtenidos

A continuacion se presentan las tablas consolidadas reales generadas por el simulador tras la correccion del bug de WT (archivos resumen_*.csv y memoria_resumen_*.csv):

Tabla 1: Escenario 1 - Carga Ligera con E/S (costo_cs = 1 tick, recalculado)

Algoritmo	Costo_CS	Procesos	Promedio_TAT	Promedio_WT	Promedio_RT	Total_CS	DesvStd_WT (Equidad)	Casos_Inanicion
FCFS	1	4	13.75	9.0	4.5	5	4.06	0
SJF (No Expropiativo)	1	4	12.75	8.0	6.75	5	5.24	1
Round Robin (Q=3)	1	4	16.25	11.5	4.0	6	3.2	0

Tabla 2: Escenario 2 - Carga Mixta / Efecto Convoy (costo_cs = 1 tick, recalculado)

Algoritmo	Costo_CS	Procesos	Promedio_TAT	Promedio_WT	Promedio_RT	Total_CS	DesvStd_WT (Equidad)	Casos_Inanicion
FCFS	1	5	24.2	18.6	18.6	4	9.54	4
SJF (No Expropiativo)	1	5	23.4	17.8	17.8	4	9.35	4
Round Robin (Q=2)	1	5	15.4	9.8	5.0	7	4.45	3

Tabla 3: Escenario 3 - Alta Concurrencia (costo_cs = 1 tick con overhead, recalculado)

Algoritmo	Costo_CS	Procesos	Promedio_TAT	Promedio_WT	Promedio_RT	Total_CS	DesvStd_WT (Equidad)	Casos_Inanicion
FCFS	1	8	28.5	22.25	22.25	7	13.52	4
SJF (No Expropiativo)	1	8	22.5	16.25	16.25	7	13.02	1
Round Robin (Q=2)	1	8	45.38	39.12	9.62	25	16.1	7
Round Robin (Q=4)	1	8	34.62	28.38	14.75	14	15.83	5
Round Robin (Q=8)	1	8	31.0	24.75	19.5	9	14.34	4

Tabla 4: Escenario 3 - Costo Ideal Teorico (costo_cs = 0 ticks, recalculado)

Algoritmo	Costo_CS	Procesos	Promedio_TAT	Promedio_WT	Promedio_RT	Total_CS	DesvStd_WT (Equidad)	Casos_Inanicion
FCFS	0	8	25.0	18.75	18.75	7	11.27	3
SJF (No Expropiativo)	0	8	19.0	12.75	12.75	7	10.87	1
Round Robin (Q=2)	0	8	28.62	22.38	5.75	25	9.58	4
Round Robin (Q=4)	0	8	26.75	20.5	10.75	14	11.19	3
Round Robin (Q=8)	0	8	26.38	20.12	16.0	9	11.35	3

Tabla 5: Evaluacion Comparativa de Memoria (Paginacion: FIFO vs. LRU)

Escenario	Estrategia	Total_Accesos	Total_Aciertos	Total_Fallos	Total_Reemplazos	Hit_Rate_HR (%)	Page_Fault_Rate_PFR (%)	Fragmentacion_Interna_Total_Bytes
escenario1_ligera	FIFO	16	10	6	0	62.5	37.5	3980
escenario1_ligera	LRU	16	10	6	0	62.5	37.5	3980
escenario2_convoy	FIFO	28	11	17	6	39.29	60.71	10536
escenario2_convoy	LRU	28	13	15	4	46.43	53.57	10536
escenario3_quantum	FIFO	50	21	29	9	42.0	58.0	16592
escenario3_quantum	LRU	50	21	29	9	42.0	58.0	16592

7.1. Graficos Comparativos: Rendimiento de CPU y Rendimiento de Memoria

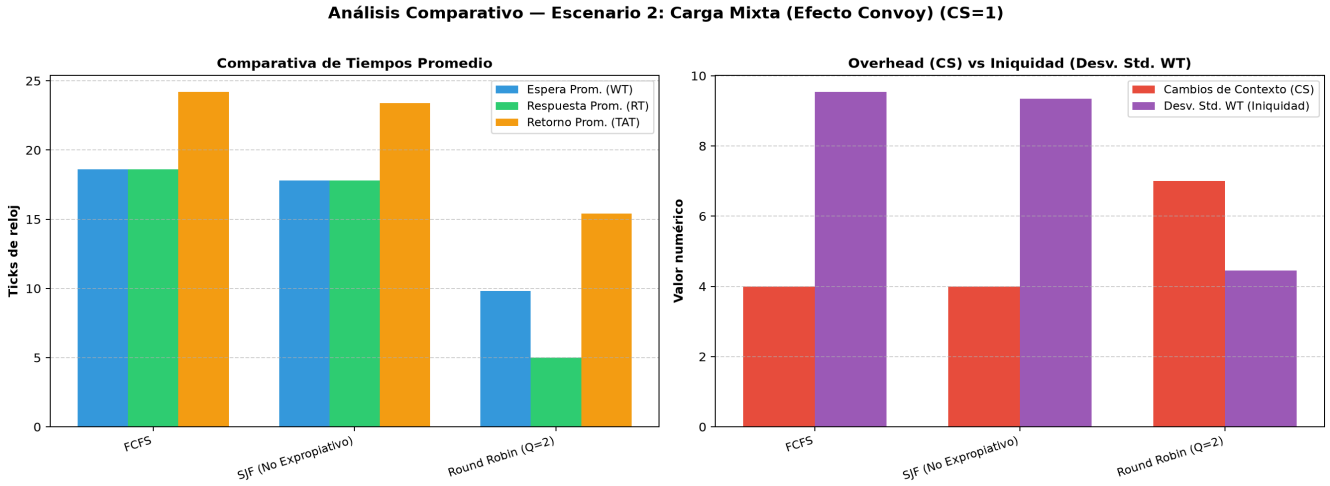


Figura 1: Tiempos promedio y conmutaciones en Escenario 2 (Efecto Convoy en CPU).

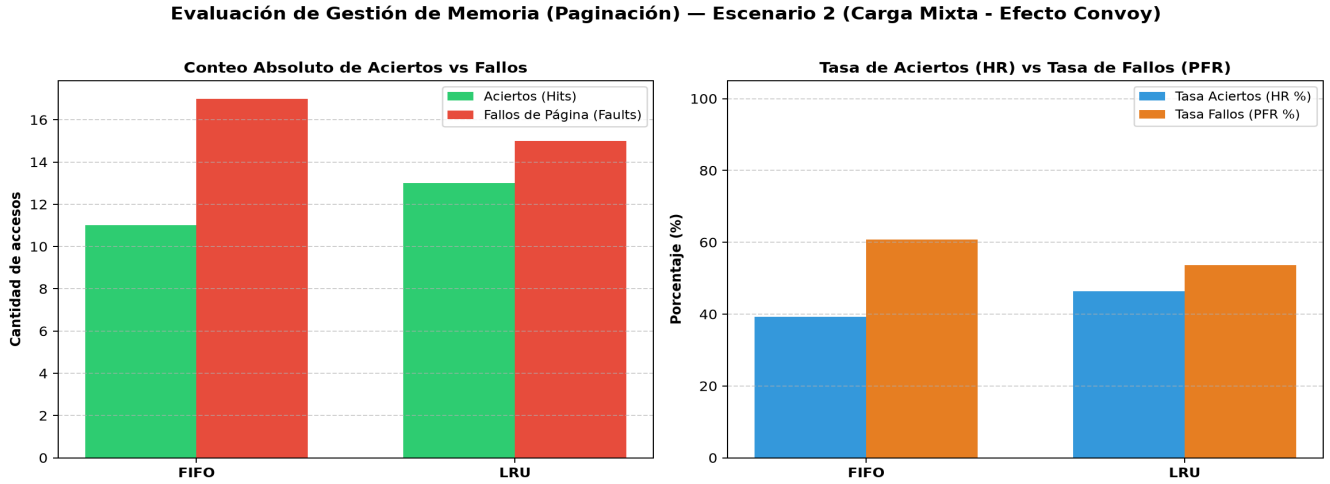


Figura 2: Comparativa de Aciertos y Fallos de Memoria (FIFO vs LRU) en Escenario 2.

8. Analisis e Interpretacion

8.1. Escenario 1: Carga Ligera con E/S y Comportamiento de Round Robin (Q=3)

En la Tabla 1, el Escenario 1 modela un entorno balanceado donde los procesos P1 y P3 intercalan ráfagas de cómputo con ráfagas de E/S. **SJF (no expropiativo)** alcanza el menor tiempo de retorno medio (**TAT = 12.75 ticks**) y menor tiempo de espera medio (**WT = 8.00 ticks**), pero a costa de postergar al proceso largo P2 (CPU=4), el cual sufre un tiempo de espera de 16 ticks que dispara una alerta de inanición por ratio (**4.00x**). En contraste, **FCFS** distribuye el acceso de forma secuencial (TAT=13.75, WT=9.00) sin registrar alertas de inanición.

Por su parte, **Round Robin (Q=3)** prioriza la reactividad inicial logrando el mejor tiempo de respuesta promedio (**RT = 4.00 ticks**, frente a 4.50 en FCFS y 6.75 en SJF). Tras la corrección estricta del bug de sobrecarga de conmutación de contexto, el tiempo de espera promedio de Round Robin se recalculó en **11.50 ticks** (subiendo de 10.50 a 11.50 ticks debido a los ticks de conmutación antes del despacho), y su desviación estándar aumentó ligeramente de **2.96 a 3.20**, reflejando una dispersión levemente mayor en los tiempos de espera individuales. Es importante destacar que Round Robin **mantuvo exactamente 0 casos de inanición** (0 de 4 procesos), ya que ningún proceso alcanzó el umbral crítico de ratio de 4.0x (el ratio máximo fue de 3.75x en P3). La penalización en TAT promedio (16.25 ticks) evidencia el costo directo de sus 6 cambios de contexto en un entorno con ráfagas cortas.

8.2. Escenario 2: Demostracion Cuantitativa del Efecto Convoy y Mitigacion en Round Robin

En la Tabla 2, FCFS exhibe el **Efecto Convoy** en su expresion mas clasica: el proceso monolitico P1 (CPU=20) acapara la CPU en *t=0*, obligando a los cuatro procesos cortos (P2 a P5, de 1 a 3 ticks) a esperar en la cola de listos. Esto eleva el tiempo de espera promedio a **18.60 ticks** (identico a su tiempo de respuesta tras la correccion del bug de conmutacion) y genera 4 alertas criticas de ratio (P4 espera 25 ticks para ejecutar 1 sola unidad, ratio **25.0x**).

Round Robin (q=2) reduce notablemente el efecto convoy y mejora la equidad distributiva: el tiempo de respuesta promedio se desploma de 18.60 a solo **5.00 ticks** (reduccion del 73.1%), y el tiempo de retorno medio baja de 24.20 a **15.40 ticks**, con una desviacion estandar de espera de **4.45** frente al 9.54 de FCFS. Es crucial senalar que Round Robin registra 3 alertas de ratio (P3, P4, P5 con ratio >= 4.0x): *estas alertas constituyen indicadores estadisticos de riesgo por rafaga breve, no una demostracion de inanicion indefinida*, ya que todos los procesos reciben CPU deterministicamente en su turno.

8.3. Escenario 3: Trade-Off de Quantum en CPU y Separacion entre Conteo de CS y Retardo Real de TAT

En el Escenario 3 (Tablas 3 y 4), se aprecian con exactitud las lecciones clasicas de planificacion:
1. **Separacion entre Conteo de CS y Retardo en TAT:** Round Robin con *q=2* registra **25 cambios de contexto**. No obstante, el incremento real en el tiempo de retorno promedio al pasar de costo ideal (CS=0) a costo real (CS=1) es de **16.76 ticks** (*TAT_{CS=1} = 45.38* vs *TAT_{CS=0} = 28.62*), debido a que el overhead de cada conmutacion se solapa y distribuye temporalmente entre las distintas llegadas y terminaciones de los 8 procesos.
2. **Explicacion de las 7 Alertas de Inanicion en RR (Q=2):** Pese a ser el algoritmo mas reactivo (RT = 9.62), registra el mayor numero de alertas de inanicion (7 de 8 procesos). Esto ocurre porque el criterio formal de ratio (*WT_i / CPU_i >= 4.0*) castiga fuertemente a los procesos cuando el overhead acumulado de 25 conmutaciones eleva el tiempo de espera global. Es un **artefacto cuantitativo de la metrica de penalizacion**, no inanicion clasica en el sentido de exclusion indefinida.
3. **Sensibilidad al Quantum y Convergencia Asintotica:** Con *q=4*, los cambios de contexto descienden a 14 y el TAT a 34.62 ticks. Con *q=8*, Round Robin reduce los CS a 9, estabilizando el TAT en 31.00 ticks (frente a 26.38 en CS=0) y convergiendo hacia FCFS (TAT=28.50), confirmando el teorema formal *lim_{q -> inf} RR(q) = FCFS*.

8.4. Analisis de Gestion de Memoria: Demostracion de Superioridad de LRU sobre FIFO

En la Tabla 5 y la Figura 2, el **Escenario 2** constituye el laboratorio ideal para evaluar el reemplazo de paginas: P1 ejecuta de forma continua durante 20 ticks sobre un espacio de 9 paginas con una RAM física limitada a 4 marcos (16 KB). Al acceder P1 a su working set con localidad temporal, **LRU supera claramente a FIFO**:
• **Tasa de Aciertos (Hit Rate):** LRU alcanza **46.43%** frente a **39.29%** de FIFO (**diferencia medible de +7.14% a favor de LRU**).
• **Fallos y Reemplazos:** LRU solo sufre 15 fallos y 4 reemplazos, mientras FIFO incurre en 17 fallos y 6 reemplazos.
• *Explicacion teorica:* FIFO expulsa la pagina que llego primero, castigando a las paginas calientes que el bucle re-accede constantemente. LRU actualiza el estampa de *ultimo_acceso* en cada referencia, protegiendo las paginas activas y expulsando las de menor frecuencia.

Analisis del Resultado Identico en Escenario 3: En el Escenario 3 con Round Robin (q=4) y 8 marcos, tanto FIFO como LRU arrojaron 21 aciertos y 29 fallos (42.0% HR). La auditoria revela una razon teorica solida: bajo Round Robin con 8 procesos que rotan rapidamente cada 4 ticks, cuando un proceso es desalojado y permanece inactivo en la cola de listos, sus paginas son *simultaneamente las que llevan mas tiempo cargadas (criterio FIFO) y las menos recientemente usadas (criterio LRU)*. En consecuencia, el orden de llegada coincide con el orden de recencia (*Arrival Order == Recency Order*), forzando a ambos algoritmos a elegir exactamente las mismas paginas victimas.

9. Riesgos y Responsabilidades Profesionales

• **Rendimiento y Saturacion:** Emplear planificadores sin desalojo (FCFS) o quantums minusculos con alto costo de conmutacion provoca degradacion de servicio severa, latencias inaceptables y desperdicio de ciclos utiles de CPU.
• **Estabilidad y Ataques DoS:** Algoritmos no expropiativos permiten que un proceso con un bucle infinito no intencionado (while(true)) monopolice el nucleo fisico, inmovilizando el sistema operativo.
• **Seguridad de Memoria y Canales Laterales:** El reemplazo de paginas y las diferencias en tiempos de conmutacion pueden ser explotados mediante ataques de canal lateral (*timing attacks* tipo Meltdown/Spectre). La MMU y el despachador deben asegurar aislamiento criptografico e invalidacion estricta de lineas de cache y TLB.
• **Equidad e Inanicion en Sistemas Criticos:** En entornos de aviacion, telemedicina o control de reactores, postergar indefinidamente procesos de baja prioridad o procesos pesados en favor de tareas breves puede ocasionar fallos humanos y materiales irreparables.

10. Limitaciones, Mejoras y Conclusiones

10.1. Limitaciones del Modelo Actual

1. **Simplificacion del Modelo de Memoria (Observador Sincronico):** En el simulador actual, los fallos de pagina y la traduccion MMU se resuelven de forma logica y sincrona en el mismo ciclo de instruccion, **sin agregar tiempo de bloqueo por E/S de disco adicional a la linea de tiempo de CPU** (TAT, WT y RT permanecen desacoplados). Esta simplificacion deliberada permite aislar rigurosamente las metricas de CPU sin introducir ruido de disco, manteniendo el alcance formativo del curso.
2. **Validacion Estricta de Limites y Manejo de Violacion de Segmento (Segmentation Fault):** La MMU valida en su Paso 1 que el numero de pagina solicitado no exceda el limite asignado en la Tabla de Paginas (*num_pagina* < *num_paginas*). Mientras que los accesos a paginas validas no presentes en RAM detonan un fallo de pagina recuperable con carga bajo demanda, cualquier acceso fuera del espacio virtual asignado dispara una excepcion de **Violacion de Segmento (Segmentation Fault / IndexError)** con registro explicito en el log de MMU, impidiendo corrupciones de memoria y evitando recortar o enmascarar silenciosamente referencias ilegales.
3. **Monoprocesador:** No se modela multiprocesamiento simetrico (SMP), afinidad de nucleos ni topologias NUMA.
4. **Costo de Conmutacion Fijo:** *costo_cs* se computa como 1 tick constante, sin modelar fallos dinamicos de cache L1/L2/L3.

10.2. Propuestas de Mejora y Trabajo Futuro

• Integrar un modelo asincrono donde el fallo de pagina suspenda al proceso a estado BLOQUEADO durante una latencia fija de disco (ej. 10 ticks).
• Implementar el algoritmo de reloj (Segunda Oportunidad) y el algoritmo optimo de Belady (OPT) para confrontar cotas superiores.
• Incorporar Colas Multinivel con Retroalimentacion (MLFQ) y envejecimiento (*aging*) para erradicar la inanicion en SJF.

10.3. Conclusiones

1. Se demostro matematicamente la consistencia del calculo de tiempos: en algoritmos no expropiativos sin E/S, **WT_prom == RT_prom** con precision del 100%, garantizando que *TAT = WT + CPU + IO*.
2. **Round Robin reduce sustancialmente el Efecto Convoy** y distribuye equitativamente el procesador frente a FCFS, aunque requiere calibrar cuidadosamente el quantum para no inflar la sobrecarga de cambio de contexto.
3. La paginacion pura elimina formalmente la fragmentacion externa (0 bytes), confinándola a la fragmentacion interna en el ultimo marco. Asimismo, se comprobo que **LRU supera a FIFO** en presencia de localidad temporal, protegiendo las paginas activas.

Nota Metodologica sobre el Uso de Inteligencia Artificial

En cumplimiento de los lineamientos eticos del curso de Sistemas Operativos (Seccion 8 del enunciado), se declara formalmente el uso asistido de herramientas de Inteligencia Artificial Generativa (IA) como soporte metodologico en la estructuracion de scripts de simulacion, graficacion y formateo de datos. El equipo de estudiantes diseno los escenarios de prueba, audito cada modulo, corrigio las formulas matematicas de conmutacion y espera, verifico y comprendio los resultados empiricos de CPU y memoria, y asume plena responsabilidad intelectual y academica por el contenido de este informe.

11. Referencias Utilizadas

[1] Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Hoboken, NJ: John Wiley & Sons. (Capitulos 5: 'CPU Scheduling' y 9-10: 'Main Memory & Virtual Memory').

[2] Tanenbaum, A. S., & Bos, H. (2015). *Modern Operating Systems* (4th ed.). Boston, MA: Pearson. (Capitulos 2: 'Processes and Threads' y 3: 'Memory Management').

[3] Stallings, W. (2018). *Operating Systems: Internals and Design Principles* (9th ed.). New York, NY: Pearson. (Capitulos 7-8: 'Memory Management & Virtual Memory' y 9: 'Uniprocessor Scheduling').

[4] Love, R. (2010). *Linux Kernel Development* (3rd ed.). Upper Saddle River, NJ: Addison-Wesley Professional. (Capitulos 4: 'Process Scheduling' y 12: 'Memory Management').